Field Hours

CSCE 625 Artificial Intelligence

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 45 minutes

Search

An agent is a state, a set of actions, and a goal test. Uninformed search expands states in an order that ignores the particular goal, except for the test. Breadth-first search expands the shallowest state first and returns a shortest path in hops when every action costs the same. Its memory holds the whole frontier. Depth-first search holds only the path it is on, and it can miss a shallow goal if the space is deep or cyclic. Use a visited set when actions can loop.

Uniform-cost search expands the cheapest path first. It is Dijkstra on the implicit graph of states, and it needs nonnegative action costs. It finds an optimal path. Breadth-first search is the special case where every action costs one.

A-star adds a heuristic h(n), an estimate of the remaining cost. The priority is g(n) plus h(n), where g is the cost already paid. If h never overestimates, A-star is optimal. A heuristic that overestimates can return a pretty path that is not cheapest, because the algorithm may finish a tempting branch before a dull but better one. Manhattan distance on a grid with no obstacles is admissible. Straight-line distance is admissible for travel when the cost is at least the crow-flies distance.

Write the frontier on paper for any example smaller than ten states. If you cannot list the order of expansion, you do not yet know which search you implemented. The rest of the course, games and planning included, is this lecture with a different definition of state.

A-star expands the lowest f = g + h. h must not overestimate.

1. Pay g to reach the state

2. Estimate h to the goal

3. Expand smallest g + h

4. Stop at the goal

A grid, start at (0,0), goal at (2,1)

1. Actions move one cell north, south, east, or west, cost 1. Manhattan distance is the heuristic.

2. Start f is 0 + 3 = 3. East to (1,0) has g 1 and h 2, so f is 3. North to (0,1) has g 1 and h 2, so f is 3.

3. Expand east, then the cell (2,0) has f = 2 + 1 = 3, and (1,1) has f = 2 + 1 = 3. The goal (2,1) is reached at g = 3.

4. Optimal cost is 3. An overestimate that said h(0,0) = 9 would still be a number, and it would not be safe.

Problem 1. All action costs are 1. Is breadth-first search optimal? Is depth-first search optimal?

Answer. Breadth-first search is optimal because the first time it generates the goal, no shorter hop-path remains on the frontier. Depth-first search is not. It can walk a long branch and hit the goal before it ever tries a one-step branch.

Problem 2. Heuristic h returns 0 for every state. What algorithm is A-star then, and is it optimal?

Answer. f equals g. A-star becomes uniform-cost search. With nonnegative costs it is optimal. The heuristic is admissible because 0 never exceeds the true remaining cost.

Lecture 2 · 40 minutes

Games

A two-player zero-sum game is a tree of moves. Max plays, then Min, then Max, until a leaf with a utility for Max. Minimax says Max chooses the child with the largest value and Min chooses the child with the smallest value. The value of a node is defined only after the children are defined, so the implementation is a depth-first walk. On a tree with branching factor b and depth d the walk looks at about b to the d leaves. That is the whole problem with the algorithm, and it is why chess programs do not expand to the end of the game.

Alpha-beta pruning walks the same tree and keeps two bounds. Alpha is the best value Max can already force. Beta is the best value Min can already force. If a node's remaining children cannot change the decision above, they are not generated. The result equals minimax. The savings depend on the order. If the best move is tried first, the effective branching is about the square root of b, which doubles the depth you can see for the same work.

When the tree is too deep, replace the leaf utility with an evaluation function and stop at a fixed depth. The function is a guess, and every flaw in it becomes a move. A function that counts material and ignores whose turn it is will walk into a one-move loss looking rich. Horizon effects are the same bug at a distance: a bad event sits one ply past the cutoff, so the search pretends it is not there.

Ethics shows up here as the utility, not as a paragraph at the end. If the leaf score rewards only winning, the agent will take a legal move that harms a person the score does not mention. Changing the legal moves, or putting that harm in the utility, changes the game. A sermon after the algorithm does not.

One ply of a game. Max and Min read the same tree and optimize opposite ways.

Max move -> Min move -> Max move -> Leaf score

A tree with four leaves

1. Root is Max. Two children, each a Min node. Min on the left has leaves 3 and 5. Min on the right has leaves 2 and 9.

2. Left Min returns 3. Right Min returns 2. Max returns 3.

3. Alpha-beta, left first: after the left Min returns 3, alpha is 3. The right Min sees 2 first. 2 is already worse for Max than 3, so the leaf 9 is not examined.

Problem 1. Same tree, but the right Min's leaves are 4 and 9, and you still search left first. Does alpha-beta skip any leaf?

Answer. Left Min returns 3, so alpha is 3. Right Min's first leaf is 4, which is better than 3, so Min must look at 9 to know its own value. Min returns min(4, 9) = 4. Max then picks 4. No leaf is skipped.

Problem 2. Evaluation at depth 2 says a position is +8, but the opponent can force a loss in one more ply. What is the name of this failure?

Answer. The horizon effect. The cutoff hid the loss. The repair is a deeper search on unstable positions, or a different evaluation, not a claim that +8 was optimal play.

Lecture 3 · 45 minutes

Logic and planning

Propositional logic is a language for facts that are true or false. A model assigns each symbol a bit. A sentence is satisfiable when some model makes it true, valid when every model does. Inference algorithms exist because you do not want to list the models. Resolution is refutation-complete for propositional logic: if a set of clauses is unsatisfiable, resolution will derive the empty clause. It is not a way to generate interesting consequences in a helpful order. You still have to ask a question.

First-order logic adds objects and quantifiers. It can say 'every lock has a key' without naming them. The price is that entailment is undecidable. This course uses it as the language of a planner, not as a theorem-proving marathon. If you cannot write the sentence, you do not have the constraint, no matter how sophisticated the search is.

Classical planning is search in a factored state. A state is a set of ground facts. An action has preconditions and effects that add and delete facts. Forward search from the initial state works, and it is the search lecture again, with a heuristic that ignores the delete lists. Ignoring deletes makes the problem easier and the estimate admissible for the remaining number of actions, because you only become more able to act.

A plan that is correct in the model and disastrous outside it is still a correct plan in the model. Write the preconditions you actually care about, including the ones that protect a person or a machine. The planner will not invent a precondition you refused to name.

A planner is search over sets of facts. The action is legal only if the preconditions hold.

1. State: a set of facts
2. Action needs preconditions
3. Add and delete effects
4. Stop at the goal facts

A locked drawer

1. Facts: Closed, Locked. Goal: Open. Key is in hand already, so HasKey is true and stays true.
2. Unlock requires Locked and HasKey. Effect: delete Locked.
3. Open requires Closed and not Locked. Effect: delete Closed, add Open.
4. The only legal plan is Unlock, then Open. Open first has no model in which its precondition holds.

Problem 1. You drop the precondition HasKey because the planner 'should be smart'. What plan does forward search return, and what is wrong with it?

Answer. Unlock then Open, even when HasKey is false. The plan is legal in the model you wrote and impossible on the drawer you own. The missing precondition is the bug.

Problem 2. Is a sentence that is true in every model satisfiable, valid, both, or neither?

Answer. Both. Valid means true in every model. Satisfiable means true in at least one. Every valid sentence is satisfiable. The converse is false.

Lecture 4 · 45 minutes

Probability and decisions

A probability is a number between 0 and 1 assigned to events so that the whole space sums to 1. Conditional probability P(A given B) is P(A and B) divided by P(B). Bayes' rule turns that around when the thing you can measure is the evidence and the thing you want is the cause. The denominator is the probability of the evidence, summed over the causes. Skip the denominator and you have scores, not probabilities.

A Bayesian network is a directed acyclic graph of conditional independences. A node is independent of its non-descendants given its parents. That is the whole modeling claim. The joint distribution factors into one conditional table per node. For n binary nodes with at most k parents, you store about n times 2 to the k numbers instead of 2 to the n. The savings are the point. The graph is not a picture of causality unless you earned that reading separately.

A Markov decision process is a state, actions, transition probabilities, and a reward. A policy picks an action in each state. The value of a state under a policy is the expected sum of rewards, usually discounted by gamma so a reward now is worth more than the same reward later. Value iteration applies the Bellman update until the numbers stop moving. With a finite state space and a discount below 1, it converges.

The reward is the ethics. If the reward pays for speed and does not charge for a broken machine or a harmed person, value iteration will break the machine. Put the cost in the reward before you iterate. An agent that is optimal for the wrong number is not a neutral piece of mathematics.

A tiny network. Alarm depends on fire. Calls depend on alarm, not on fire directly.

Fire

Alarm, given fire

Neighbor calls, given alarm

One Bayes update

1. P(fire) = 0.01. P(alarm given fire) = 0.9. P(alarm given no fire) = 0.1.

2. P(alarm) = 0.9 * 0.01 + 0.1 * 0.99 = 0.009 + 0.099 = 0.108.

3. P(fire given alarm) = 0.009 / 0.108 = 1/12, about 0.083.

4. The alarm raised the chance from 1 percent to about 8 percent. It did not make the fire likely.

Problem 1. Value iteration, one state, one action, reward 4 every step, discount 1/2. What is the value?

Answer. V = 4 + (1/2) V. So (1/2) V = 4 and V = 8. The infinite series 4 + 2 + 1 + 1/2 + ... sums to 8.

Problem 2. In the alarm numbers, what is P(fire given no alarm)?

Answer. P(no alarm given fire) = 0.1. P(no alarm) = 1 - 0.108 = 0.892. P(fire and no alarm) = 0.1 * 0.01 = 0.001. The conditional is 0.001 / 0.892, about 0.00112. The fire is now less likely than the base rate.

Lecture 5 · 40 minutes

Learning inside an agent

Learning here means updating a function from examples when the model is too large to write by hand. A supervised example is an input and a label. The hypothesis is a function from inputs to labels. Training picks the hypothesis that scores well on the examples you have. Testing asks about examples you did not train on. A perfect training score with a bad test score is memorization. Report the test score.

A linear classifier scores a weighted sum of features and thresholds it. The features are the modeling decision. A feature you did not include cannot be used, just as a planner cannot obey a precondition you did not write. Add a feature when you can say what it measures. Do not add one because the training score went up on thirty examples.

Overfitting is fitting noise. Regularization adds a cost for large weights so the hypothesis stays smoother than the examples strictly require. A separate validation set chooses the strength of that cost. Using the test set to choose it makes the test set a second training set, and the number you publish is optimistic.

An agent that learns online changes its policy while acting. The examples are then produced by the policy, not by the world at large. A bad early policy collects bad examples and can confirm itself. That is a technical failure, and it is also the ethics problem: the data the system sees are the data its behavior created. Hold out a test that the policy did not generate before you claim the agent learned the task.

The same labeled points. One hypothesis will still be wrong on the next point.

Memorize

Zero training error

A kink per point

Test error unknown

Linear rule

May miss a point

Two numbers to learn

Test can be estimated

Three points, one feature

1. Examples: x = 0 label 0, x = 1 label 0, x = 2 label 1.

2. Threshold 'predict 1 when x is at least 2' fits all three.

3. Threshold 'predict 1 when x is at least 1.5' also fits all three. The training set cannot tell them apart.

4. A test point at x = 1.7 decides. You do not get to pick it after seeing it and still call it a test.

Problem 1. Training accuracy is 100 percent and test accuracy is 54 percent on a balanced two-class problem. What happened?

Answer. The hypothesis memorized the training sample. 54 percent is barely above a coin flip. Collect more data, simplify the hypothesis, or regularize. Do not ship the training number.

Problem 2. You tune a regularizer on the test set until test accuracy peaks. Why is the published test accuracy biased?

Answer. The test set was used to choose the model, so it is no longer unseen. The peak is an optimistic draw over the choices you tried. A fresh set, or a three-way split, is the repair.

Lecture 6 · 180 minutes

Constraints

A constraint satisfaction problem is a set of variables, a finite domain for each, and a set of constraints that mention subsets of the variables. A solution is an assignment that satisfies every constraint. Map coloring, scheduling, and sudoku are the same problem with different constraints. Saying so is useful only if you write the variables and the domains down.

Backtracking assigns one variable at a time and retreats when a constraint is already violated. Forward checking, when you assign a variable, deletes values from its neighbors that are now impossible. Arc consistency is stronger: a value survives in a domain only if some value in the other variable's domain supports it. AC-3 enforces that on every arc. It does not by itself search. It shrinks domains before and during search.

Variable ordering changes the size of the tree. The minimum-remaining-values heuristic assigns the variable with the smallest domain next, on the grounds that it is the most likely to fail soon. Least-constraining value tries values that leave the neighbors the most options. These are heuristics. They do not change which solutions exist.

If the constraint graph is a tree, arc consistency plus a pass from the leaves solves the problem in time linear in the number of values times the number of edges. Cycles are why the general problem is NP-complete. A tree decomposition of small width is the structured way out, and it is the reason a schedule with local constraints can be easy while an arbitrary one is not.

Search with propagation.

Pick a variable -> Try a value -> Delete unsupported neighbor values -> Backtrack if a domain empties

Two variables, domains of size two

1. X and Y each have domain {1, 2}. The only constraint is X + Y = 3, so the allowed pairs are (1, 2) and (2, 1).

2. Arc consistency on X: 1 is supported by Y = 2, and 2 is supported by Y = 1. Nothing is deleted. The same holds for Y.

3. Assign X = 1. Forward checking deletes Y = 1, because 1 + 1 is not 3. Y's domain is {2}. No search past that assignment is required.

Problem 1. Arc consistency leaves every domain nonempty. Is the problem guaranteed to have a solution?

Answer. No, once the constraint graph has a cycle. Three variables, each domain {1, 2}, and constraints that they are pairwise different, is arc consistent if you are not careful with the count: actually pairwise different on {1, 2} for three variables is not arc consistent, because a value needs a support in each neighbor simultaneously across arcs one at a time. A standard counterexample is a cycle of inequalities that is locally supported and globally impossible after a larger domain. Nonempty domains after AC-3 do not certify a solution. You still have to search, unless the graph is a tree.

Problem 2. Why does assigning the most constrained variable first often beat assigning variables in the order they were written?

Answer. A failure near the root throws away a larger subtree than a failure near a leaf. The variable with the smallest domain is the one most likely to fail immediately. You pay a small amount now to avoid copying a dead end across the remaining variables.

Lecture 7 · 180 minutes

Logical inference

Propositional logic is a language of atoms and the connectives not, and, or, and implication. A model is a truth assignment. Entailment means every model of the knowledge base is a model of the query. Inference rules are sound if they only derive entailed sentences, and complete if they can derive every entailed sentence. You want both. A rule that sometimes invents facts is worse than a slow one.

Resolution is a complete method for propositional entailment when the knowledge base is in conjunctive normal form. From (A or B) and (not B or C) you derive (A or C). To test whether the knowledge base entails Q, add not Q and ask whether resolution derives the empty clause. If it does, not Q is inconsistent with the knowledge base, so Q is entailed. The procedure can take exponential time. It will not lie if you implement the clause set correctly.

Forward chaining on definite clauses, which have exactly one positive literal, runs in linear time in the size of the knowledge base. It is complete for queries that are single atoms, and incomplete for arbitrary propositional queries. Backward chaining starts from the query and is the same idea from the other end. Use them when the knowledge is a set of rules. Use resolution when you need the full language.

First-order logic adds quantifiers and relations. Unification replaces propositional matching. The course needs you to see that a finite propositional encoding of a planning problem is already first-order structure flattened into atoms. It does not need a full theorem-prover project.

Refutation by resolution.

1. Put the knowledge base in CNF

2. Add the negated query

3. Resolve pairs until the empty clause

4. Or stop when no new clause appears

A three-clause knowledge base

1. Clauses: (A or B), (not A or C), (not B or C). Query: C.

2. Add (not C). Resolve (A or B) with nothing about C yet. Resolve (not A or C) with (not C) to get (not A). Resolve (not B or C) with (not C) to get (not B).

3. Resolve (A or B) with (not A) to get (B). Resolve (B) with (not B) to get the empty clause. The knowledge base entails C. A model with C false cannot satisfy all three original clauses.

Problem 1. Forward chaining on definite clauses derives C from the example above. Why is that not a reason to use forward chaining on every knowledge base?

Answer. Definite clauses have exactly one positive literal. A knowledge base with (A or B) and no other structure is not a set of definite clauses, and forward chaining as defined for rules does not apply. Resolution still does. Completeness is relative to the fragment you claimed.

Problem 2. The empty clause is derived after adding not Q. What do you report, and what would the empty clause mean if you had not added not Q?

Answer. Report that the knowledge base entails Q. If the empty clause appears before you add not Q, the knowledge base itself is contradictory, and it entails every query. You should notice that case instead of treating every question as yes.

Lecture 8 · 180 minutes

Planning

A planning problem names a set of facts, a start state, a goal, and actions. Each action has preconditions that must hold and effects that add or delete facts. A plan is a sequence of actions from the start to a state that contains the goal. This is search in a state space that you should not expand by hand, because the number of states is exponential in the number of facts.

Ignore-delete-lists is a relaxation: pretend actions never remove a fact. The relaxed problem is easier, and the number of relaxed actions needed to reach the goal is an admissible heuristic for the real search. Admissible means the heuristic never overestimates the remaining cost. A* with an admissible heuristic returns an optimal plan. An inadmissible heuristic can return a plan that works and is not the shortest.

Partial-order planning keeps actions unordered except where one action's effect is another action's precondition. The plan is a set of constraints, not a single sequence, until the end. That avoids committing to an order that a later threat will break. A threat is an action that deletes a fact another action needs. You resolve it by ordering the threat before the producer or after the consumer.

The frame problem is the obligation to say what does not change. STRIPS-style effects solve it by assuming unspecified facts stay as they are. If your action language lacks that default, every action must mention every fact, and the model is not usable.

From facts to a plan.

1. Write the start facts

2. Write each action's preconditions and effects

3. Relax deletes to get a heuristic

4. Search, and check the real deletes

A locked drawer

1. Facts: locked, have-key, drawer-open. Start: locked, have-key. Goal: drawer-open.

2. Unlock requires locked and have-key, deletes locked, adds unlocked. Open requires unlocked and not drawer-open, adds drawer-open.

3. The relaxed problem that ignores deletes still needs both actions, so the heuristic is 2. The real plan is unlock, then open. Searching with that heuristic expands the right plan first.

Problem 1. You drop the precondition 'unlocked' from Open, and the planner says the plan is a single Open. Whose mistake is that?

Answer. The modeler's. The planner answered the problem you wrote. A missing precondition is not a search bug. Read the plan back against the world before you blame the heuristic.

Problem 2. A heuristic counts the number of goal facts not yet true. On the drawer, the goal is one fact, so the heuristic is 1 at the start. Is that admissible, and is it tight?

Answer. It is admissible if each action can add at least one missing goal fact and actions cost 1, because you need at least one action per missing goal fact only when no action adds two. Here one action cannot open a locked drawer, so the true cost is 2 and the heuristic of 1 never overestimates. It is admissible and not tight. Ignore-delete-lists was tighter.

Lecture 9 · 180 minutes

Belief networks

A Bayesian network is a directed acyclic graph whose nodes are random variables, plus a conditional probability table for each node given its parents. The joint distribution factors into those tables. That factorization is the reason the model is smaller than an explicit table over every variable. A joint over n binary variables is 2^n numbers. A network with small parent sets is linear in n times the table size.

Inference by enumeration sums the joint over the variables you did not ask about. It is correct and exponential. Variable elimination sums one variable at a time and reuses factors. Its cost depends on the order. A bad order recreates the full joint. Belief propagation is exact on trees and approximate, unless you are careful, on graphs with cycles.

Conditional independence is the point of the arrows. Given its parents, a node is independent of its non-descendants. You can read some independences off the graph with d-separation, and you should not claim an independence the graph does not support. An arrow is not a causal claim unless you built the graph that way and you are willing to say what intervention means.

A network is only as good as the numbers in the tables. Invented probabilities produce invented posteriors. If the tables came from counts, say how many observations, and refuse a probability of zero unless the event is impossible rather than merely unseen.

A three-node network. Alarm depends on fire. Call depends on alarm.

Fire, no parents

Alarm given fire

Call given alarm

Joint is the product of the three tables

One update

1. P(fire) = 0.01. P(alarm | fire) = 0.9. P(alarm | no fire) = 0.1. You observe the alarm and have not modeled the call yet.

2. P(alarm) = 0.9 * 0.01 + 0.1 * 0.99 = 0.009 + 0.099 = 0.108.

3. P(fire | alarm) = 0.009 / 0.108 = 1/12, about 0.083. The alarm raised the probability from 0.01, and it did not make a fire certain, because false alarms are common relative to fires.

Problem 1. You add a node Call with P(call | alarm) = 0.7 and P(call | no alarm) = 0.01. You observe a call and no alarm reading. Does the call give information about fire?

Answer. Yes. Call is not independent of fire once you have not observed alarm, because alarm is a common parent path: fire -> alarm -> call. Observing call raises the probability of alarm and therefore of fire. If you had observed alarm, the call would be independent of fire in this graph. d-separation says so, and the calculation agrees.

Problem 2. A table contains a zero for an event you have simply never recorded. What does inference do with that zero?

Answer. It treats the event as impossible and can force a posterior to zero or make a query undefined when every explanation is ruled out. Use a small count, or an explicit model of unseen events, unless you mean impossible.

Lecture 10 · 180 minutes

Decisions over time

A Markov decision process is a set of states, actions, transition probabilities, and rewards. The Markov property says the next state depends on the past only through the current state and the action. A policy picks an action in each state. The value of a state under a policy is the expected sum of discounted rewards from that state. The discount gamma, between 0 and 1, makes a reward next step worth gamma times a reward now, and it makes the infinite sum finite.

Value iteration applies the Bellman optimality backup until the values change by less than a tolerance. The backup replaces a state's value by the best action's expected immediate reward plus gamma times the expected value of the next state. With a finite state space and gamma under 1, the iteration converges to the optimal values. The greedy policy with respect to those values is optimal.

Policy iteration alternates evaluating the current policy, which is a linear system, and improving it by one greedy step. It often takes fewer iterations than value iteration and more work per iteration. Both return the same

optimum if you run them to convergence. Stopping early returns a policy that can be suboptimal by a bound you can compute from the residual.

A grid with a pit and a goal is the whole idea. If the transition is noisy, the optimal path does not hug the pit. A deterministic planner that ignores the 0.1 chance of slipping is solving a different problem, and it will look efficient until the first slip.

One value-iteration backup.

For each state -> For each action -> Expected reward plus discounted next value -> Keep the best action

Two states

1. States Good and Bad. One action. From Good you stay in Good with reward 1. From Bad you move to Good with reward 0. Gamma is 0.5.

2. Optimal values satisfy V(Good) = 1 + 0.5 V(Good), so 0.5 V(Good) = 1, and V(Good) = 2. V(Bad) = 0 + 0.5 V(Good) = 1.

3. One backup from zeros: V(Good) becomes 1, V(Bad) becomes 0. Next backup: V(Good) = 1 + 0.5 * 1 = 1.5, V(Bad) = 0.5 * 1.5 = 0.75 on the following pass if you use the new Good value. The sequence climbs to 2 and 1. It does not jump there from a single backup.

Problem 1. Gamma is 1 and every state has a positive reward available forever. What happens to the value of an infinite horizon?

Answer. The sum diverges. Values are infinite and comparisons are meaningless. Use a discount under 1, or a finite horizon, or switch to average reward and say so. Do not run value iteration and report the last finite number as if it were the infinite sum.

Problem 2. Your robot slips into the pit 10 percent of the time when it walks beside the pit, and the pit reward is -100. The shortest path walks beside the pit for one square and a longer path does not. Why might the longer path have higher value?

Answer. The expected cost of one slip is 10, in the same units, before discounting, which can exceed the extra steps of the long path. The optimum is an expectation, not a length. A planner that only counts steps is ignoring the transition model you wrote.

Lecture 11 · 180 minutes

Learning a policy

Reinforcement learning is an MDP whose transitions or rewards you do not know as a table. You learn from trajectories. A model-based method learns the transition counts and then runs value iteration. A model-free method learns values or a policy without storing the transition table. Q-learning learns the value of a state-action pair from the backup of the next state's best action, and it does not need the transition model. It converges under conditions you should be able to name: every pair is tried infinitely often, and the learning rate falls appropriately.

Exploration is not optional. A greedy policy with respect to a half-learned value will never try the action that looked bad once. Epsilon-greedy takes a random action with probability epsilon. A fixed epsilon never stops

exploring. Decaying epsilon can stop too soon. The course requirement is that you notice the choice, not that you invent a new schedule.

A reward function is a specification. If you pay the agent for points, you get points, including points earned by a behavior you did not picture. That is specification gaming, and it is an ordinary failure of the formalization, the same kind as a planner with a missing precondition. Changing the learning rate does not fix a reward that scores the wrong event.

Policy gradient methods optimize a parameterized policy by climbing the expected return. They can represent stochastic policies and they can be unstable. This course asks you to explain Q-learning on a small table before you treat a neural policy as anything but a function approximator with a failure mode.

Two ways to learn the same task.

Model-based

Count transitions

Average the rewards

Plan with value iteration

Q-learning

Store Q(state, action)

Back up from the next max

No explicit transition table

One Q update

1. Q(s, a) is 0. You take a, receive reward 1, and land in s' where the best Q is 2. Gamma is 0.5. Learning rate is 0.5.

2. The target is 1 + 0.5 * 2 = 2. The update moves halfway from 0 toward 2, so the new Q(s, a) is 1.

3. A second identical experience moves halfway from 1 toward 2, so Q becomes 1.5. It approaches 2 and does not store the transition that produced s'.

Problem 1. Epsilon is 0 from the start and every Q is 0. What policy do you actually run?

Answer. A fixed arbitrary tie-break among actions, forever, if greedy breaks ties the same way. You never gather the data the convergence proof assumed. Epsilon-greedy with epsilon 0 is not exploration.

Problem 2. The agent is rewarded for reaching a charger and learns to drive in circles that each touch the charger. Is the learning algorithm wrong?

Answer. The algorithm optimized the reward you wrote. Circles are optimal if each touch pays and there is no cost for time or wear. Change the reward or the horizon. Do not call the optimizer broken for succeeding.

Lecture 12 · 180 minutes

State you cannot see

A hidden Markov model has a state you do not observe, a transition on that state, and an observation that depends on the state. Filtering asks for the distribution over the current state given observations so far. Prediction asks for a future state. Smoothing asks for a past state given later observations. They are different queries. A filter that you then treat as a smoothed history is answering the wrong one.

The forward step multiplies the predicted state distribution by the likelihood of the observation and normalizes. On a small discrete model you can do it with a vector. The normalize step is not optional: without it the numbers are not a distribution and the next step compounds the error.

Particle filters represent the distribution by weighted samples when the state is continuous or huge. You predict each particle through the transition, weight it by the observation, and resample. If you resample from a handful of particles that all missed the true region, the filter collapses and more observations cannot revive a state you threw away. That failure has a name. Use it when your robot is suddenly certain and wrong.

Localization and tracking in the robotics courses are this recursion with a motion model and a sensor model. The artificial-intelligence obligation is to write both models and the query. A heatmap is not a method.

One filter update.

Predict with the transition -> Weight by the observation -> Normalize -> That distribution is the input to the next step

Two states, one observation

1. States H and L, prior 0.5 each. Transitions: stay with probability 0.8, switch with 0.2. You observe High, and P(High | H) = 0.75, P(High | L) = 0.25.

2. Predict is still 0.5 if you start from 0.5 and the chain is symmetric. Weights: H gets 0.5 * 0.75 = 0.375, L gets 0.5 * 0.25 = 0.125. Sum 0.5.

3. Normalize: P(H | High) = 0.375 / 0.5 = 0.75, P(L | High) = 0.25. The observation moved the belief and did not erase the 0.25.

Problem 1. You skip the normalize step and feed 0.375 and 0.125 into the next prediction. What breaks?

Answer. They are not a distribution. Their sum is 0.5, so every later observation is scaled by a shrinking or arbitrary factor, and a comparison of hypotheses is no longer a probability. Normalization is the step that makes the filter a posterior.

Problem 2. Smoothing says the robot was probably in the hallway at time 3. Filtering at time 3 said the dock. Can both be right?

Answer. Yes. Filtering uses observations up to time 3. Smoothing also uses observations after time 3, which can rule out the dock. They are different conditional distributions. Report which one you computed.

Lecture 13 · 180 minutes

What the agent does not know

An agent that acts on a model is only as calibrated as the model. The earlier weeks built search, logic, probability, and learning. This week is the boundary. A fact absent from the knowledge base is not false. A probability of 0.9 is not a promise about the next single trial. A policy that is optimal for the MDP is optimal for that MDP, not for the building the MDP simplified.

Calibration means that among the events you called 80 percent, about 80 percent occur. A classifier can be accurate and badly calibrated, and a planner that treats a calibrated 0.8 as a certainty will take risks the model told you not to take. Reliability diagrams are a check, not a decoration.

Ethics in this course is a constraint on the specification, not a paragraph after the demo. Whose data is in the model, who is affected by a false positive, and whether the action can be refused are part of the problem statement. If you cannot say who pays for a mistake, the problem is not specified.

When the model and the world disagree, the engineering move is to measure the disagreement and narrow the claim. It is not to add parameters until the training log looks calm.

Two different claims.

What the model supports

Optimal for the stated MDP

Entailed by the stated sentences

Calibrated on the stated test

What it does not

Optimal in the building

True because it was not mentioned

Safe because the loss fell

A false positive with a cost

1. A detector flags 10 out of 100 people, and 4 of the 10 flags are wrong. Each false flag costs the person an hour. Each missed case, of which there is 1, costs a different harm you must name rather than bury inside accuracy.

2. Accuracy can be high while those 4 hours are real. Report the false-positive count beside the accuracy.

3. The specification either accepts those 4 hours or it changes the threshold. That choice is part of the agent. It is not a later ethics essay disconnected from the threshold.

Problem 1. The knowledge base does not contain a sentence about a door being locked. May the planner assume the door is unlocked?

Answer. Only if the problem statement says that unmentioned doors are unlocked. Otherwise the

door's status is unknown, and a plan that requires it to be unlocked is incomplete. Absence of a sentence is not a truth value.

Problem 2. Your probabilities are calibrated and the optimal action still harms a group that was rare in the data. Does calibration answer the objection?

Answer. No. Calibration says the numbers mean what they say. It does not say the objective was acceptable. The harm belongs in the reward or the constraint, or the system should not take that action. A well-calibrated wrong objective is still wrong.

Lecture 14 · 180 minutes

An agent you can grade

The project for this course is a single agent on a small, fully specified world. Pick one: a search problem with an admissible heuristic you can justify, a planner whose preconditions you can violate on purpose to show the failure, or an MDP whose values you compute by hand on a subset and by iteration on the rest. Do not pick all three.

The write-up has four parts. The formal problem. The algorithm and its invariant or backup. A worked instance small enough to check. And one input on which a plausible wrong version fails. That fourth part is how you show you understand the method. A demo that only shows the happy path is not gradable.

Measure something. Nodes expanded against the heuristic. Plan length against the relaxation. Value error against the number of backups. A table with one column is not a measurement.

If you use learning, separate the data you fit from the data you report. The standard from the machine-learning course applies here even though the models look like agents. An agent that memorized the test maze has not learned a policy.

What the write-up must contain.

The formal problem

The invariant or the backup

A worked instance

One input that breaks the wrong version

A heuristic you can check

1. Grid from (0, 0) to (2, 1), four-connected, each step costs 1. Manhattan distance is |dx| + |dy| = 3.

2. The true shortest path is 3. The heuristic never exceeds that on this instance, and on a grid with no obstacles Manhattan never exceeds the true cost, so it is admissible here.

3. If you add a wall and keep using Manhattan, it remains admissible because walls only lengthen the true path. It may become a bad guide. Admissible does not mean efficient.

Problem 1. Your A* heuristic cuts corners through walls and sometimes exceeds the true remaining cost. What may you no longer claim about the path it returns?

Answer. You may not claim it is shortest. A* with an inadmissible heuristic can return a feasible path that is not optimal. Drop the heuristic or repair it before you use the word optimal.

Problem 2. The worked instance has three states and your code prints values for fifty. What belongs in the write-up beside the printout?

Answer. A hand calculation of the three-state case that matches the code, plus the definition of the fifty-state instance. The printout is not evidence until a smaller case agrees with arithmetic you did yourself.